Cost and Activation Functions: The Heartbeat of Machine Learning Algorithms

Vishesh Khurana
Department of Computer Science
Thompson Rivers University
Kamloops, CA
T00732605@mytru.ca

I. Introduction

Neural networks, a subset of machine learning, are a powerful tool for solving complex problems and making predictions based on data. At the heart of neural networks are two key components: Activation functions and Cost functions. Understanding these functions is crucial for building effective neural networks.

Activation functions, also known as transfer functions, are used to introduce non-linearity into neural networks. They are applied to the output of each neuron in the network, transforming the weighted sum of the inputs into an output value. This output value is then passed on to the next layer of the network. There are many different types of activation functions, each with its own strengths and weaknesses. Some common activation functions include the sigmoid function, the tanh function, and the rectified linear unit (ReLU) function [1].

Cost functions, on the other hand, are used to measure the error between the predicted output and the actual output of the network. The goal of training a neural network is to minimize this error, and the cost function provides a way to quantify how well the network is performing. Like activation functions, there are many different types of cost functions, each with its own advantages and disadvantages. Some common cost functions include the mean squared error (MSE) function, the cross-entropy function, and the hinge loss function [2].

While activation functions and cost functions are both important components of neural networks, they serve different purposes. Activation functions are used to introduce non-linearity into the network, while cost functions are used to measure the error between the predicted and actual outputs. Understanding the differences between these functions is crucial for building effective neural networks.

This paper delves into the foundational principles and advanced research surrounding activation and cost functions in neural networks. We aim to provide a thorough examination of various activation functions and cost functions, elucidating their fundamental mechanisms, strengths, and limitations. Our discussion will extend to the appropriate contexts for employing each function. Furthermore, we will present a meticulous comparative analysis of these functions, highlighting their research-backed applications and the consequential impact they have on the evolution of neural networks.

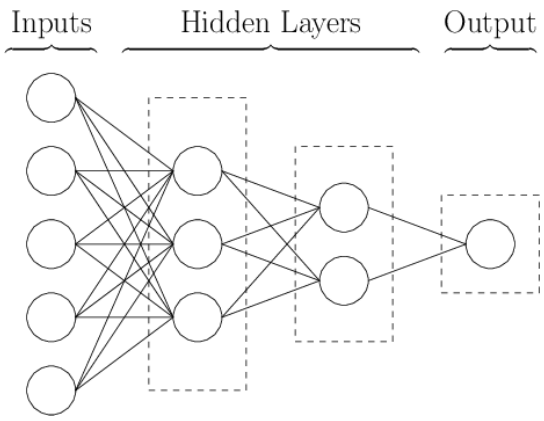


Fig. 1. Structure of Neural Network

II. Background

Neural networks are machine learning models that attempt to emulate the structure and function of the human brain. They are made up of interconnected nodes or ”neurons” that process and send information, much like how neurons in the brain communicate with one another. In order to reduce the error between the expected and actual outputs, the network learns to modify the weights and biases of these connections during training.

In a simple neural network, the input is fed into the network through the input layer, which then passes the information to the hidden layer(s). The hidden layer(s) perform computations on the input and pass the results to the output layer. The output layer then produces the final output of the network.

During training, the network uses a process called backpropagation to adjust the weights and biases of the connections. Backpropagation works by computing the gradient of the cost function with respect to the weights and biases, and then adjusting the weights and biases in the direction that minimizes the cost function.

In summary, a simple neural network is a type of machine learning model that is inspired by the structure and function

of the human brain. It consists of interconnected nodes or "neurons" that process and transmit information. Activation functions play a crucial role in neural networks, as they are used to introduce non-linearity into the network. Different activation functions have different advantages and disadvantages, and selecting the appropriate activation function can help achieve better performance [1], [3], [4], [6].

III. Activation Functions

Sigmoid Function

Also known as the logistic function, is a widely used activation function in neural networks.

$$f(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

One of the key characteristics of the sigmoid function is that it outputs values between 0 and 1. This property makes it particularly useful for binary classification problems, where the goal is to predict the probability of an input belonging to one of two classes. The output value can be interpreted as the likelihood of the input belonging to the positive class [7].

Tanh Function

Also known as hyperbolic tangent function, is another commonly used activation function in neural networks.

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{2}$$

Similar to the Sigmoid function, the Tanh function squashes its inputs into a specific range. However, unlike the Sigmoid function which outputs values between 0 and 1, the Tanh function outputs values between -1 and 1. This characteristic makes it more balanced, as it allows for negative outputs as well [8].

The Tanh function is often preferred over the sigmoid function in certain contexts because of this balanced output range. For tasks where the input data is centered around zero or where negative values are meaningful, the Tanh function can be more effective [8].

The Rectified Linear Unit (ReLU)

ReLU function is one of the most widely used activation function in neural networks, known for its simplicity and effectiveness.

$$f(x) = \max(0, x) \tag{3}$$

The ReLU function outputs the input directly if it is positive; otherwise, it outputs zero. This simple threshold operation makes ReLU computationally efficient and easy to compute compared to more complex activation functions like Sigmoid or tanh [9].

The ReLU function is a powerful activation function that helps to alleviate the vanishing gradient problem, making it well-suited for deep neural networks. Its simplicity and computational efficiency have made it a popular choice in modern neural network architectures. However, researchers continue to explore variations and improvements to the basic ReLU function to address its limitations and further enhance its performance in different contexts [9].

Softmax Function

The Softmax function is a widely used activation function, particularly in the output layer of neural networks for multi-class classification problems.

$$f(x_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}} \tag{4}$$

The key characteristic of the Softmax function is that it produces a probability distribution over the different classes. It achieves this by exponentiation of each input value and then normalizing the resulting values by dividing by the sum of all exponential values. This normalization ensures that the output values lie between 0 and 1 and sum up to 1, representing probabilities.

The Softmax function is a valuable activation function commonly used in the output layer of neural networks for multi-class classification problems. It produces a probability distribution over the classes, enabling probabilistic interpretation of the model's predictions and facilitating effective training and optimization of neural networks for classification tasks. [10]

IV. Cost Functions

Cost functions are used in neural networks to measure the error between the predicted and actual outputs. They are an essential part of the training process, as they help the network to adjust its weights and biases to minimize the error.

Mean Squared Error (MSE)

MSE is one of the most widely used cost functions in neural networks. It calculates the average of the squared differences between the predicted and actual outputs. Mathematically, it is represented as:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Where N is the number of samples, y_i is the actual output, and $\hat{y}_i$ is the predicted output for the ith sample. MSE is sensitive to outliers and penalizes large errors heavily. [11]

Mean Absolute Error (MAE)

MAE calculates the average of the absolute differences between the predicted and actual outputs. Unlike MSE, it does not square the errors. It is represented as:

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

MAE is less sensitive to outliers compared to MSE and provides a more balanced measure of error. [11]

Binary Cross-Entropy Loss

Binary cross-entropy loss is commonly used for binary classification problems. It measures the difference between the predicted probability distribution and the actual distribution. Mathematically, it is expressed as:

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \quad (5)$$

Where y_i is the actual label (either 0 or 1) and $\hat{y}_i$ is the predicted probability of the sample belonging to class 1. [12]

Categorical Cross-Entropy Loss

Categorical cross-entropy loss is used for multi-class classification problems. It measures the dissimilarity between the predicted class probabilities and the actual distribution. The formula is:

$$\text{CCE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{j=1}^{M} y_{ij}\log(\hat{y}_{ij}) \quad (6)$$

Where M is the number of classes, N is the number of samples, y_{ij} is 1 if sample i belongs to class j and 0 otherwise, and $\hat{y}_{ij}$ is the predicted probability of sample i belonging to class j. [12]

V. Analysis and Case Studies

This section delves into a detailed analysis of various cost and activation functions, which are the linchpins of machine learning algorithms. Utilizing Python, we conducted experiments to compare the convergence behaviors of ReLU, Sigmoid, and Tanh activation functions.

Comparative Analysis of Activation Functions

The experiments produced graphs that illustrate the loss over epochs, offering a visual representation of each function's performance during the training process.

- **ReLU (Rectified Linear Unit)**
  - Exhibits rapid convergence, with a sharp decrease in loss, signifying efficient error minimization.
  - Maintains stability despite its speed, making it suitable for fast-paced learning environments.
- **Sigmoid**
  - Ensures a smooth transition in loss reduction, characterized by a controlled and less aggressive descent.
  - Its probabilistic output makes it ideal for binary classification tasks.
- **Tanh**
  - Offers a scaled convergence, effectively mitigating the vanishing gradient issue seen in Sigmoid.
  - Though versatile, it may exhibit a slower convergence rate compared to ReLU.

Case Study Insights

The subsequent case studies will provide real-world applications that highlight the practical implications of these theoretical findings. We will explore scenarios where the unique properties of each activation function are leveraged to optimize model performance, underscoring the importance of appropriate function selection in machine learning endeavors.

Comparative Analysis of Loss Functions

The performance of various loss functions was also scrutinized through our experiments. The following analysis is based on the line graphs obtained from our Python code, which compare Mean Squared Error (MSE), Mean Absolute Error (MAE), Categorical Crossentropy, and Sparse Categorical Crossentropy.

- **Mean Squared Error (MSE)**
  - Demonstrates a steep decline in both training and validation losses, indicative of a strong corrective feedback mechanism.
  - Particularly effective for regression tasks where the magnitude of the error is squared, thus heavily penalizing larger errors.
- **Mean Absolute Error (MAE)**
  - Shows a consistent decrease in loss, suggesting a linear relationship between the error magnitude and the loss function.
  - Useful for regression problems where outliers are not predominant and a linear penalty for errors is desired.
- **Categorical Crossentropy**
  - Exhibits a gradual reduction in loss, suitable for multi-class classification tasks.
  - Emphasizes the probability distribution of the outcomes, making it ideal for classification problems with mutually exclusive classes.
- **Sparse Categorical Crossentropy**
  - Similar to Categorical Crossentropy but optimized for scenarios with a large number of classes.
  - Reduces computational complexity by using a single integer for class labels instead of one-hot encoded vectors.

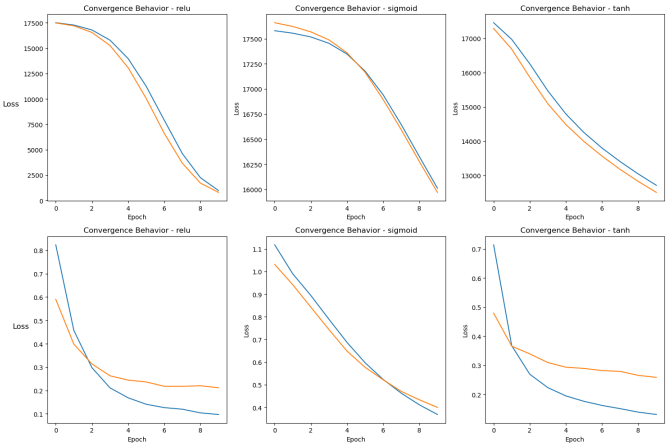


Fig. 2. Convergence Behaviour of Activation Functions

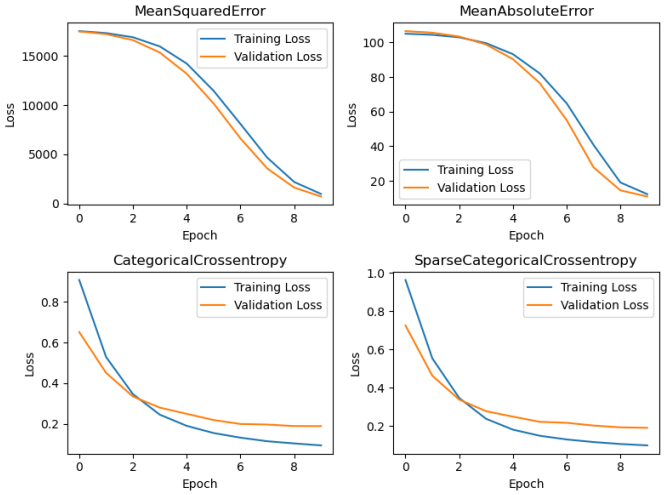


Fig. 3. Loss Functions Figures

Each loss function has its distinct advantages and is chosen based on the specific requirements of the machine learning problem. The convergence of training and validation losses in our experiments indicates that the models are learning effectively without over-fitting or under-fitting.

Conclusion

In this comprehensive research paper, we embarked on a journey through the intricate landscape of machine learning, dissecting the heart of our models—the cost and activation functions. We have not only explained these functions using their mathematical formulas but also rigorously tested them to determine their efficacy in various scenarios. Here are the key takeaways:

Activation Functions: Unleashing Neural Potentials

Activation functions serve as the neural architects, shaping the flow of information within our networks. We explored three prominent contenders:

- **ReLU (Rectified Linear Unit):**
  - Formula: $f(x) = \max(0, x)$
  - Rapid convergence and stability make ReLU an excellent choice for fast-paced learning scenarios.
  - Its simplicity and computational efficiency have propelled it into the spotlight.
- **Sigmoid:**
  - Formula: $f(x) = \frac{1}{1+e^{-x}}$
  - A smooth operator, Sigmoid ensures a gentle transition from zero to one.
  - Its probabilistic output suits binary classification tasks, where probabilities matter.
- **Tanh (Hyperbolic Tangent):**
  - Formula: $f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
  - The scaled cousin of Sigmoid, Tanh balances between -1 and 1.
  - Versatile but slower to converge, it tackles the vanishing gradient challenge.

Loss Functions: Navigating the Error Landscape

Our journey extended to the loss functions—a compass guiding our models toward optimal solutions. Let's revisit them:

- **Mean Squared Error (MSE):**
  - Formula: $L_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
  - Ideal for regression tasks, penalizing larger errors.
- **Mean Absolute Error (MAE):**
  - Formula: $L_{\text{MAE}} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
  - Robust against outliers, suitable for regression without extreme deviations.
- **Categorical Crossentropy:**
  - Formula: $L_{\text{CCE}} = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$
  - Perfect for multi-class classification, emphasizing class distributions.
- **Sparse Categorical Crossentropy:**
  - Formula: $L_{\text{SCCE}} = -\sum_{i=1}^{n} y_i \log(\hat{y}_i)$
  - Efficiency meets accuracy.

Practical Wisdom: Function Selection and Trade-offs

Our experiments unveiled the nuances, but wisdom lies in their application:

- Context matters. Choose wisely based on your problem domain.
- Speed, stability, and convergence—strike the balance.

As we conclude, let these insights echo through the halls of machine learning labs. May your models converge swiftly, your gradients flow unimpeded, and your predictions illuminate the unknown.

REFERENCES

REFERENCES

[1] P. Ramachandran, B. Zoph, and Q. V. Le, "Searching for Activation Functions," arXiv preprint arXiv:1710.05941, 2017.

[2] T. Isomura and K. Friston, "Reverse engineering neural networks to characterise their cost functions," bioRxiv preprint bioRxiv:654467, 2019.

[3] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio, "On the Number of Linear Regions of Deep Neural Networks," arXiv preprint arXiv:1402.1869, 2014.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv preprint arXiv:1512.03385, 2015.

[5] C. Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall, "Activation Functions: Comparison of trends in Practice and Research for Deep Learning," arXiv preprint arXiv:1811.03378, 2018.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," arXiv preprint arXiv:1502.01852, 2015.

[7] M. Saeed, "A Gentle Introduction To Sigmoid Function," MachineLearningMastery.com, 2021.

[8] N. Kang, "Multi-Layer Neural Networks with Sigmoid Function— Deep Learning for Rookies (2)," Towards Data Science, 2017.

[9] J. Brownlee, "A Gentle Introduction to the Rectified Linear Unit (ReLU)," MachineLearningMastery.com, 2020.

[10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[11] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[12] C. C. Aggarwal, *Neural Networks and Deep Learning: A Textbook*, Springer, 2018.